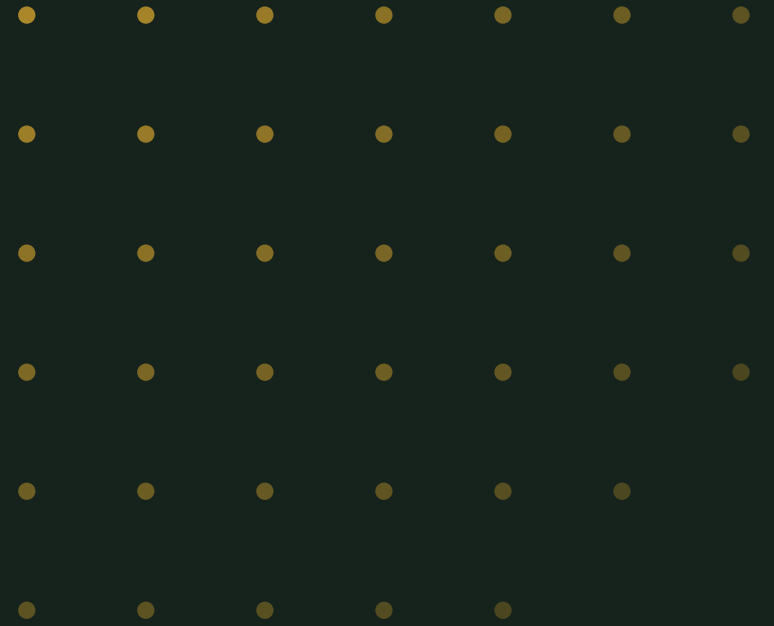


# Generation Is Cheap. Judgment Isn't.

Craig Shyjak · Prairie Dev Con 2026





**I've always been the person who takes things apart just to see how they work. I'm not an early adopter — most days I'm the skeptic in the room. But most of what I build now starts as a prompt.**

20+ years writing code, still hands-on. This isn't a hype pitch.



**The scary part isn't AI writing bad code. It's AI writing plausible code nobody understands well enough to own.**

## OPENING

# Four questions, one transition

1

### How We Build

Review becomes production

2

### How We Break

Understanding becomes scarce

3

### How We Defend

Trust boundaries become explicit

4

### How We Grow

Judgment becomes the career



**You used AI. You shipped faster. The demo worked.  
Then production broke — and the uncomfortable  
question wasn't “what did the model do?” It was  
“who here understands this well enough to own it?”**

01 / 04

# How We Build

*Output is cheap. Review becomes production.*





**The keyboard-and-mouse paradigm isn't dead —  
but typing code is no longer the default constraint.**

## Judgment, not speed

# 20:1

---

the coding-time ratio Sackman, Erikson & Grant found in 1968 — echoed by study after study since. The difference was never just typing. It was problem framing, judgment, and knowing what not to build.

- If everyone has a 10x multiplier, the multiplier becomes the floor — and judgment becomes the ceiling.
- GitClear: refactoring dropped from 25% to under 10% of all changes (2021–2024); duplicated code up 8x in a year.
- “The best code is no code at all.” — Jeff Atwood

## Cheap in effort. Not free.

# \$1,000

---

in tokens per engineer per day — StrongDM's self-reported benchmark for a functioning “software factory”

- “Generation is cheap” means cheap in effort and time — the token bill is real, and it doesn't disappear because the code did.
- The better question: are we buying output, or buying back time to decide what is worth building?
- “Speed without direction is just acceleration.”



**Output scales faster than trust. The platform that hosts the code is becoming the platform that manages the agents around it.**

## Review is the bottleneck

3.6x

---

growth in GitHub merged pull requests since Jan 2023 —  
the review queue is now the constraint

- Godot banned autonomous AI agents — reviewer burnout, a mentorship pipeline that dies on a model.
- Zig: review is “contributor poker.” You invest in the person, not the diff.
- curl killed a six-year bug bounty program after AI-generated reports overwhelmed the signal.



**Generation is cheap. Ownership is not.**

02 / 04

# How We Break

*Understanding becomes scarce while fragility compounds*



# Observability has to answer questions

*We're not short on telemetry. We're short on systems that make the next action obvious.*

1

## Start with the first questions

not the dashboards you already have

2

## Tie metrics to action

owner, runbook, escalation

3

## Instrument decisions

not only infrastructure

4

## Budget for alert fatigue

if nobody acts, the signal is wrong

5

## Make context queryable

search is not enough at 3am

HOW WE BREAK



**Not who knows how to ask an AI to find it. Who knows how to find it when the AI is wrong.**

## HOW WE BREAK

# The most dangerous metric is how productive it felt

# 19% slower

---

measured — yet developers estimated they were 20% faster (METR RCT)

- Predicted beforehand: 24% faster.
- Felt afterward: 20% faster.
- Actually was: 19% slower.
- Feeling faster is not the same thing as being faster.

## You can't practice a skill you've delegated.

17%

---

lower comprehension for the AI-assisted group — and the biggest gap was debugging

- 52 professional developers, learning a new async library (Anthropic, 2026)
- When the tool fixes the error for you, it can short-circuit the loop that builds debugging intuition.
- That's where production needs humans most.

## HOW WE BREAK

# Vendor benchmarks point to the same risk

**+23.5%**

---

incidents per pull request, year over year  
(Cortex)

**+30%**

---

change failure rate over the same period  
(Cortex)

**1.7x**

---

more review issues in AI-authored PRs vs.  
human-written (CodeRabbit)

## HOW WE BREAK

# When AI writes, reviews, and diagnoses

*Something is broken. The logs are noisy. The metrics disagree. Everyone has been in that room.*

- **AI wrote the code.**

Now the same class of system is being asked to investigate when it breaks — AWS DevOps Agent, Dynatrace Davis AI, and Datadog Bits AI SRE are all converging on that pitch.

- **Independence becomes an architecture problem**

When the investigator shares the builder's blind spots, the fix is not “less AI.” It is clearer ownership, stronger ground truth, and human sign-off where the blast radius matters.

- **The question changes**

does this create an independent signal — or another confident summary of the same bad assumptions?

03 / 04

# How We Defend

*Trust boundaries become explicit*



## The asymmetry is the point

56%

---

of AI-generated code was secure in Veracode's 2026 benchmark; roughly 44% introduced a known vulnerability

- 87% of audited commercial codebases contained at least one open source vulnerability (Black Duck OSSRA 2026)
- Mean open source vulnerabilities per audited codebase: 581
- Defense depends on pattern recognition built over years — and AI lowers the offensive bar faster than it grows the defensive bench.

## HOW WE DEFEND

# The toolchain is now part of the attack surface



## There are two different “no” answers

### Quality-based

I don't trust this output yet.

### Values-based

I don't believe the socio-economic cost is worth the perceived gains.

## Trust is bigger than code quality

# \$130B

---

in data center construction blocked or delayed by  
community opposition, Jan–Mar 2026 alone

- Opposition groups have more than doubled to 833, active across 49 states
- 52% of game industry professionals say generative AI is hurting their field, up from 30% a year ago
- A developer can believe the code works and still reject the system that produced it.



***Linux is not one of those anti-AI projects.***



LINUS TORVALDS · JULY 2026



**The boundary is not “can the AI do this.” It is “who owns the risk when it is wrong.”**

04 / 04

# How We Grow

*Judgment becomes the career. So does the psychological cost.*





**Use AI to make exploration cheap. Use specialists to decide what survives contact with reality. That was never just an engineering call — it was always a product call.**



**The next generation is not entering the same field we entered. Some are learning with AI everywhere. Some are being told not to touch it. Both groups will inherit the same production systems.**

## Satisfaction and distrust are rising together

50%+

---

report higher satisfaction — while 96% don't fully trust the AI output they're shipping (Sonar, 2026)

- AI-generated code: 42% of commits today, 65% by 2027
- Only 48% always verify before committing
- 24% of the week goes to checking, fixing, validating



**Organizations often treat every minute saved as a minute available for more work. That is how a productivity gain turns into a burnout engine.**



*I've spent my whole career getting good at writing code.  
Now AI does it faster. What was this all for?*

*The years were not wasted. They trained the judgment the  
model does not have.*

## What intentional looks like

The useful question is not “should we use AI?” It is “where do we need human judgment, independent signal, and explicit accountability?”

1

Individuals: keep the reps alive

read code, debug, trace failures, decide what not to build

2

Teams: review intent, not just diffs

make ownership and assumptions visible before merge

3

Leaders: protect judgment time

do not convert every saved minute into more throughput

4

Systems: draw trust boundaries

make ground truth, blast radius, and sign-off explicit



**Judgment becomes the career. The craft did not disappear — it moved up the stack.**

CLOSE



**Not anti-AI. Not all-in. Intentional.**

CLOSE

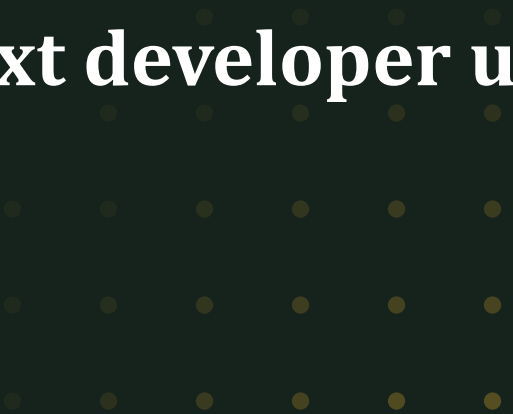
*If your AI tooling went dark tomorrow —*

**Could you still find the problem?**

**Could you still fix it?**

**Could you still ship the next feature?**

**Could the next developer understand why?**



CLOSE

**AI writes it. AI reviews it. AI diagnoses it.**

**If that's the endstate, is it the one you would choose?**

**And if not — what are you doing about it when you get  
back to work?**



CONNECT

# Continue the conversation

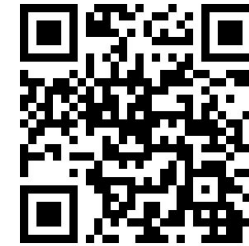
## Craig Shyjak

LinkedIn: [linkedin.com/in/craigshyjak](https://www.linkedin.com/in/craigshyjak)

Posts: agentic AI, software architecture, engineering leadership

Slides + sources: [github.com/cshyjak/talks](https://github.com/cshyjak/talks)

PDF, sources, speaker notes, and follow-up links will live there.



LinkedIn



GitHub

Connect + get the slides, sources, and follow-up links

## SOURCES

# Build & review attribution

Sackman, Erikson & Grant — 1968 exploratory study behind the 20:1 programmer-time variance claim.

GitClear 2025 AI Assistant Code Quality Report — 211M changed lines; refactoring share decline and duplication risk.

GitHub Octoverse / GitHub activity data — merged PR volume growth since 2023.

StrongDM Software Factory post — self-reported \$1,000/day/engineer token-cost benchmark.

Godot, Zig, curl maintainer statements — review burden, AI-agent policy, low-signal report pressure.

## SOURCES

# Break & operations attribution

METR RCT, 2025 — 16 experienced OSS developers, 246 real tasks; predicted +24%, felt +20%, measured -19%.

Anthropic skill-formation RCT, 2026 — 52 developers learning a new async library; lower comprehension, largest gap in debugging.

Cortex State of Engineering / Change Failure data — +23.5% incidents per PR and +30% change-failure-rate signal.

CodeRabbit analysis — 1.7x more review issues in AI-authored PRs vs. human-authored PRs.

AWS DevOps Agent, Dynatrace Davis AI, Datadog Bits AI SRE — vendor examples of AI diagnosis / investigation tooling.

## SOURCES

# Defend attribution

Veracode GenAI Code Security, 2026 — 150+ LLMs, 80 tasks; ~56% secure-code pass rate, ~44% known-vulnerability rate.

Black Duck OSSRA, 2026 — 87% of audited codebases had vulnerabilities; mean of 581 vulnerabilities per codebase.

Boost Security / TeamPCP / Aqua Trivy action reporting — timeline and supply-chain incident framing.

Microsoft Defender guidance, Mar. 24, 2026 — detection and defense guidance for compromised GitHub Actions.

Data Center Watch, 2026; GDC State of the Game Industry, 2026 — community opposition / values-based trust context.

## SOURCES

# Grow & close attribution

Sonar State of Code Developer Survey, 2026 — 1,100+ developers; 42% AI-authored committed code today, 65% expected by 2027.

Sonar verification gap, 2026 — 96% do not fully trust AI output; 48% always verify before committing; 24% of the week spent checking/fixing/validating.

Craig Shyjak LinkedIn post series and [github.com/cshyjak/talks](https://github.com/cshyjak/talks) — framing examples, follow-up notes, slides, and sources.

Prairie Dev Con 2026 keynote listing — conference context and speaker bio.

Closing synthesis is original framing, not a third-party statistic.